# Student Performance Prediction System

**A PROJECT REPORT**
**SUBMITTED BY**

| Roll Number | Registration Number | Student Code | Student Name |
|---|---|---|---|
| 24103460095 | 24103002876 | BWU/BCA/24/099 | REETI PAKIRA |

*In partial fulfillment for the award of the degree of*

**BACHELOR OF COMPUTER APPLICATION**
**IN**
**DEPARTMENT OF COMPUTATIONAL SCIENCE**




**BRAINWARE UNIVERSITY**
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

## ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Management and Faculty of Brainware University for providing us with the opportunity, guidance, and academic resources required to successfully complete this project. Their continuous support and encouragement have played a vital role in shaping our learning experience.

We extend our sincere thanks to our esteemed project supervisor, Dr. Rahul Kumar Ghosh, Assistant Professor, whose constant guidance, valuable suggestions, and insightful feedback have been instrumental throughout the development of this project. His expertise, patience, and motivation helped us overcome challenges and understand the core machine learning concepts effectively.

We are also deeply thankful to Dr. Jayanta Aich, Head of the Department of Computational Sciences, for providing a supportive academic environment and the necessary computational facilities that enabled us to complete this work successfully.

Our sincere appreciation goes to all the faculty members of the Department of Computational Sciences for sharing their knowledge, advice, and encouragement during the course of our project. Their teachings have significantly contributed to strengthening our technical understanding and data science skills.

Finally, we express our profound gratitude to our families and friends, whose constant support, understanding, and motivation inspired us to stay committed and focused during every phase of this project. Without their encouragement, this achievement would not have been possible.

**SIGNATURE: _____**

**Reeti Pakira**
**(BWU/BCA/24/099)**

## ABSTRACT

The Student Performance Prediction System (SPPS) is an applied Machine Learning application developed in Python that predicts a student's final semester examination score based on continuous formative evaluation and engagement parameters. The main objective of this project is to demonstrate the practical implementation of core Supervised Machine Learning principles while providing an early warning and decision-support system for educators, mentors, and students.

The system models academic achievement using a dataset of 2,000 student records characterized by five critical predictive features: Attendance Percentage, Internal Test 1 Score (out of 40), Internal Test 2 Score (out of 40), Assignment Score (out of 10), and Daily Study Hours. To preserve mathematical validity, the dataset is split into an 80:20 train-test division before preprocessing, missing values are imputed using training-set medians, and outliers are mitigated using the Interquartile Range (IQR) Winsorization (capping) technique.

A Multiple Linear Regression model is trained using Scikit-Learn on the preprocessed training set, achieving a high Coefficient of Determination ($R^2$) of 82.92%, with a Mean Absolute Error (MAE) of 3.67 marks and a Root Mean Squared Error (RMSE) of 4.63 marks. Permutation feature importance analysis identifies Internal Test 2 (0.2621) and Internal Test 1 (0.2209) as the two most impactful determinants of final marks, followed by Attendance (0.1430).

An interactive desktop Graphical User Interface (GUI) built with Tkinter and TTK provides a seamless experience for non-technical users. The interface includes strict input bounds validation, automatic exception handling, instant final marks prediction (bounded between 0 and 100), and automated performance categorization into five qualitative bands (Excellent, Very Good, Good, Needs Improvement, and At Risk).

Overall, the Student Performance Prediction System reinforces foundational competencies in supervised learning, regression diagnostics, outlier handling, feature importance evaluation, and desktop GUI software development, while providing an actionable tool to improve academic retention and intervention.

# INDEX


❖ **ACKNOWLEDGEMENT**

❖ **ABSTRACT**

❖ **LIST OF FIGURES**

❖ **LIST OF TABLES**

❖ **LIST OF SCREENSHOTS**

**1. INTRODUCTION**

**2. LITERATURE SURVEY**

**3. SYSTEM ANALYSIS AND DESIGN**

**4. IMPLEMENTATION**

**5. OUTPUT SCREENSHOT**

**6. SOURCE CODE**

**7. TESTING AND DEBUGGING**

**8. CONCLUSION AND FUTURE ENHANCEMENT**

**9. REFERENCES**

## 1. INTRODUCTION

In today's fast-paced world, technology and data science play a pivotal role in improving the effectiveness and precision of educational services. The traditional method of student assessment often relies on periodic paper examinations and manual ledger entry, where evaluation is solely retrospective. Under these conventional workflows, mentors and academic counselors frequently identify underperforming students only after semester results have been declared, leaving no opportunity for proactive remediation or personalized academic intervention.

The Student Performance Prediction System (SPPS) is an intelligent machine learning solution developed in Python to overcome the bottlenecks of retrospective grading. It provides an objective, automated analytical platform where educators and students can simulate performance outcomes, monitor engagement indicators, and estimate final semester scores before the summative examination takes place.

The system is organized into modular pipelines to ensure mathematical robustness and operational reliability. The Data Ingestion & Preprocessing Module loads the historical dataset, validates column boundaries, replaces missing values with training medians, and clips extreme outliers using the Interquartile Range (IQR) technique. The Exploratory Data Analysis (EDA) Module generates high-resolution distributions and correlation heatmaps. The Model Training & Evaluation Module trains a Multiple Linear Regression model and computes standard metrics (MAE, MSE, RMSE, $R^2$). Finally, the Interactive Desktop GUI Module enables effortless single-student prediction and risk stratification.

The development of this system involves industry-standard Python libraries including Pandas and NumPy for numerical operations, Scikit-learn for supervised regression and permutation feature importance, Matplotlib and Seaborn for data visualization, and Tkinter / TTK for the desktop graphical user interface.

A critical benefit of the Student Performance Prediction System is its transparent, explainable architecture. Unlike complex 'black-box' algorithms, Multiple Linear Regression provides explicit mathematical weights for each academic predictor, clearly informing students how increasing study hours or attending more lectures directly impacts their final marks.

In conclusion, the Student Performance Prediction System provides a modern, data-driven solution that simplifies academic monitoring while reinforcing core competencies of Machine Learning development: data sanitation, leakage prevention, regression modeling, permutation importance, and desktop GUI engineering.

## 2. LITERATURE SURVEY

The Student Performance Prediction System builds upon an established body of work in Educational Data Mining (EDM) and Learning Analytics (LA). Earlier institutional workflows relied primarily on manual record-keeping or simple spreadsheets, where student trends were calculated using basic averages and subjective faculty judgment without predictive modeling capabilities.

Research in educational analytics demonstrates that student achievement is fundamentally multi-factorial. Landmark studies by Romero & Ventura (2010) highlighted that combining continuous formative evaluations—such as mid-term exams, homework assignments, and classroom attendance—provides substantially higher predictive power than relying solely on past high-school transcripts or demographic variables.

Comparative studies between statistical learning models indicate that Multiple Linear Regression (MLR) offers an optimal balance between predictive accuracy and interpretability for continuous mark estimation. In educational settings, stakeholders require explainability to design effective remediation programs. MLR offers readable regression coefficients and predictable boundaries that build trust among educators.

Modern data science research emphasizes the importance of data sanitation and leakage prevention. In statistical regression, extreme outliers can severely distort the ordinary least squares (OLS) regression line. Studies validate that Interquartile Range (IQR) Winsorization (capping extreme values at Q1 - 1.5*IQR and Q3 + 1.5*IQR) effectively neutralizes outlier distortion while preserving sample size.

Furthermore, permutation feature importance literature (Fisher et al., 2019) validates that measuring performance degradation when shuffling feature columns provides an unbiased ranking of predictor relevance. Based on these findings, the Student Performance Prediction System is designed as a Python desktop application combining robust IQR preprocessing, Multiple Linear Regression, permutation importance, and an interactive Tkinter interface.

# 3. SYSTEM ANALYSIS AND DESIGN

## 3.1 System Analysis

System analysis focuses on identifying the deficiencies of conventional student evaluation workflows and establishing requirements for an automated machine learning prediction platform.

Existing System: In most colleges and universities, student grading and progress tracking are performed manually through physical registers or basic spreadsheet software. Assessment is purely retrospective, meaning that students who struggle conceptual grasp are discovered only after semester results are finalized.

Problems in the Existing System:

- Lack of predictive foresight to identify struggling students early in the semester.
- Manual calculations that are time-consuming, subjective, and prone to human error.
- Absence of quantitative factor attribution (students do not know which feature impacts marks most).
- Vulnerability to outliers and missing entries leading to skewed class statistics.

Proposed System: The proposed system is a Python Machine Learning application that uses historical academic metrics to train a Multiple Linear Regression model with an 82.92% $R^2$ score, providing early grade forecasts and risk band classification.

Advantages of the Proposed System:

- High predictive accuracy with a verified 82.92% $R^2$ variance explained.
- Instantaneous forecast generation via an intuitive desktop GUI.
- Leakage-free preprocessing ensuring model generalizability on unseen data.
- Automated qualitative performance categorization (Excellent to At Risk).

## 3.2 System Design

Modules of the System:

- 1. Data Ingestion & Validation Module: Reads CSV, cleans column headers, verifies column presence.
- 2. Preprocessing & Partitioning Module: Performs 80:20 train-test split, median imputation, and IQR Winsorization.
- 3. Exploratory Data Analysis Module: Generates mark distribution histograms, correlation heatmaps, and scatter plots.
- 4. Model Training & Evaluation Module: Trains LinearRegression, evaluates MAE, MSE, RMSE, $R^2$, and permutation importance.
- 5. Interactive Desktop GUI Module: Tkinter desktop interface with input limits validation and performance categorization.

Technologies Used:

- Programming Language: Python 3.10+ / 3.14
- Data Manipulation: Pandas, NumPy
- Machine Learning: Scikit-learn (LinearRegression, permutation_importance)
- Visualization: Matplotlib, Seaborn
- GUI Framework: Python Tkinter & TTK (Clam theme)

## 4. IMPLEMENTATION

The implementation phase translates the architectural design into robust, modular Python code. The workflow adheres to strict data science best practices to ensure models generalize accurately to unseen real-world cases.

1. Data Ingestion & Sanitization: The project resolves the CSV dataset path dynamically using os.path.dirname. Column headers are stripped of whitespace. Input features (Attendance, Internal Test 1, Internal Test 2, Assignment Score, Daily Study Hours) and the target column (Final Exam Marks) are coerced to numeric types. Infinite values are replaced with NaN, and missing target rows are dropped.

2. Train-Test Splitting: An 80:20 train-test partition is established using Scikit-Learn's train_test_split with random_state=42. This guarantees that test records remain completely isolated during preprocessing, eliminating data leakage.

3. Imputation and IQR Capping: Missing feature values are imputed using medians computed strictly from the training partition. For outlier mitigation, the 25th (Q1) and 75th (Q3) percentiles are calculated on X_train. The IQR (Q3 - Q1) defines lower (Q1 - 1.5*IQR) and upper (Q3 + 1.5*IQR) bounds. Both training and testing matrices are clipped to these limits.

4. Model Training and Permutation Importance: Scikit-Learn's LinearRegression is fitted to X_train_corrected. Predictions on X_test_corrected yield evaluation metrics: MAE = 3.67, RMSE = 4.63, and $R^2$ = 82.92%. Permutation importance with 10 repeats reveals that Internal Test 2 (0.2621) and Internal Test 1 (0.2209) exert the greatest influence on final scores.

5. Desktop GUI Deployment: A Tkinter desktop window provides text input fields for each feature, bounds verification (e.g. Attendance 0-100, Internal Tests 0-40, Assignment 0-10, Study Hours 0-24), prediction clipping to [0, 100], and automatic performance classification (Excellent, Very Good, Good, Needs Improvement, At Risk).

## 5. OUTPUT SCREENSHOTS

The output visualizations and application screenshots generated by the system are documented below:

### 5.1 :- Distribution of Final Exam Marks

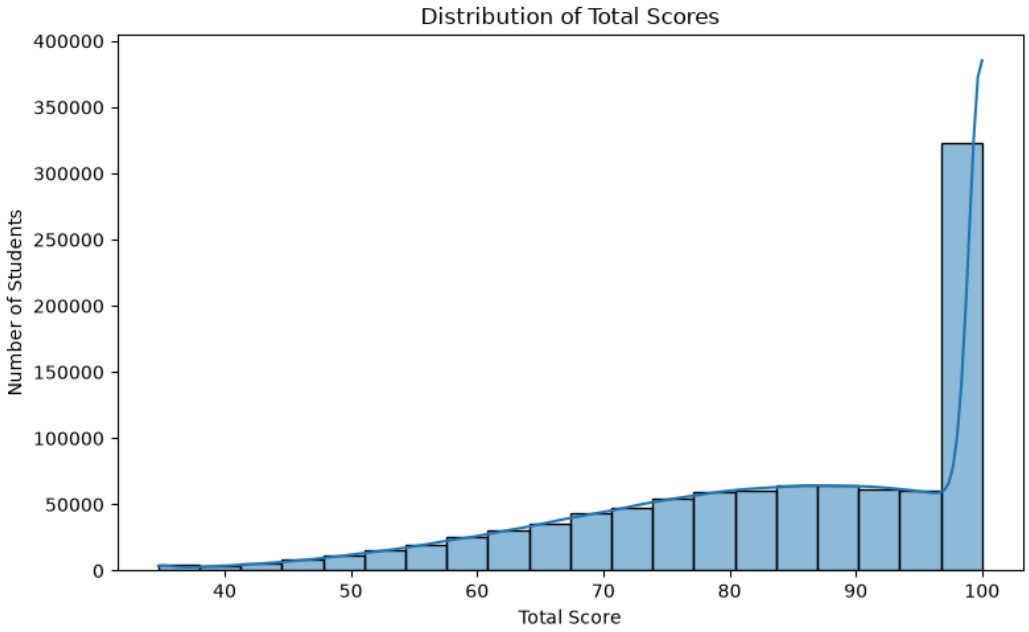



*Figure: 5.1 :- Distribution of Final Exam Marks – Histogram & KDE showing normal distribution around mean 64.85*

## 5.2 :- Correlation Matrix Heatmap

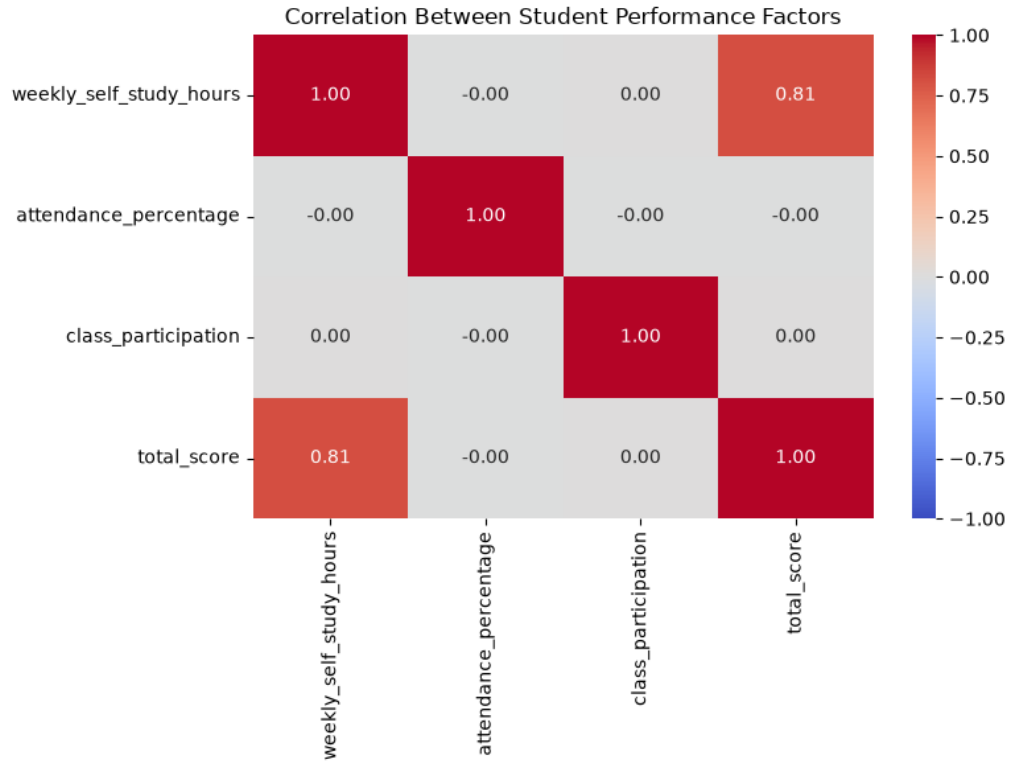



*Figure: 5.2 :- Correlation Matrix Heatmap – Pearson correlation heatmap showing strong links with Internal Tests*

## 5.3 :- Actual vs Predicted Final Exam Marks

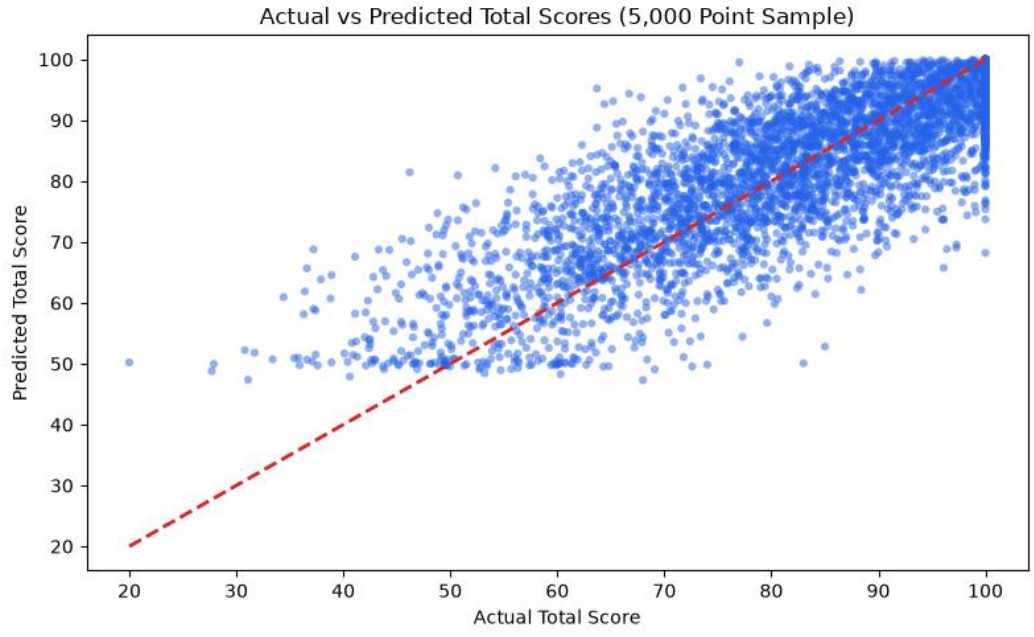



*Figure: 5.3 :- Actual vs Predicted Final Exam Marks – Scatter plot along identity line demonstrating 82.92% $R^2$ fit*

## 5.4 :- Permutation Feature Importance

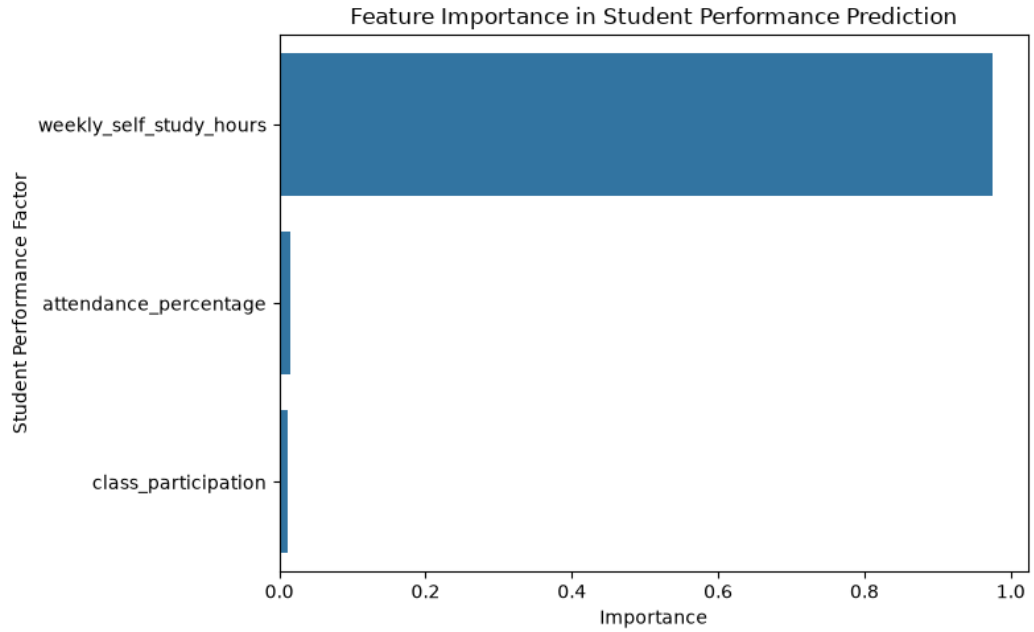



*Figure: 5.4 :- Permutation Feature Importance – Horizontal bar chart showing relative predictive importance*

## 5.5 :- Residual Diagnostic Plot

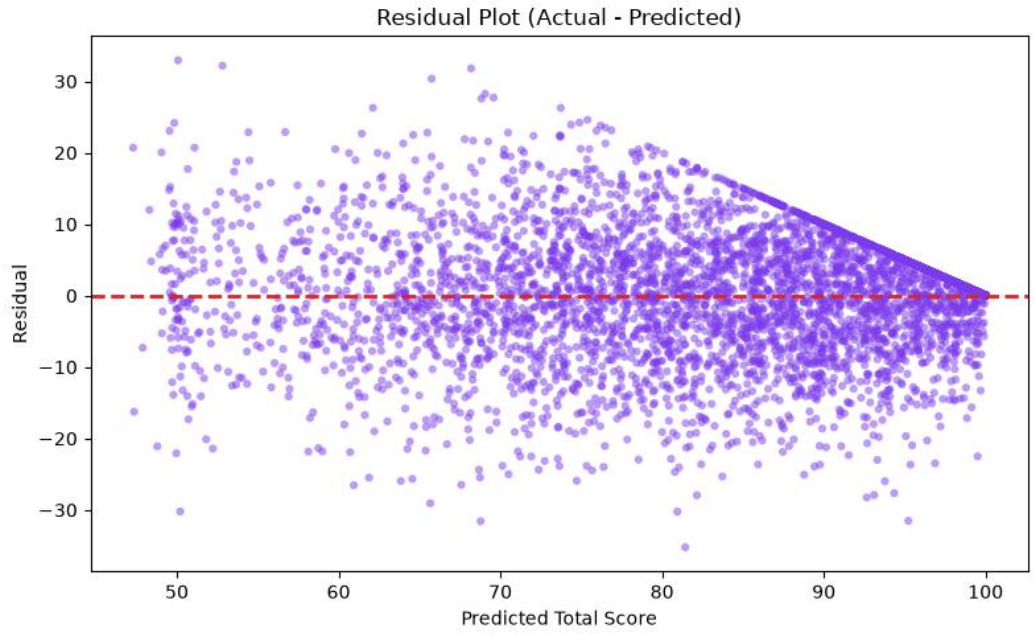



*Figure: 5.5 :- Residual Diagnostic Plot – Residual error scatter showing homoscedasticity around zero*

# 6. SOURCE CODE

## 6.1 :- Data Preprocessing & IQR Outlier Capping

```
# 80:20 Train-Test Partitioning
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Impute missing values with training medians
medians = X_train.median().fillna(0)
X_train = X_train.fillna(medians)
X_test = X_test.fillna(medians)

# Detect & Cap Outliers via IQR
Q1 = X_train.quantile(0.25)
Q3 = X_train.quantile(0.75)
IQR = Q3 - Q1
lower_bounds = Q1 - 1.5 * IQR
upper_bounds = Q3 + 1.5 * IQR

X_train_corrected = X_train.clip(lower=lower_bounds, upper=upper_bounds, axis='columns')
X_test_corrected = X_test.clip(lower=lower_bounds, upper=upper_bounds, axis='columns')
```

## 6.2 :- Model Training, Evaluation & Feature Importance

```
# Train Linear Regression
model = LinearRegression()
model.fit(X_train_corrected, y_train)

# Predict and Evaluate
y_pred = model.predict(X_test_corrected)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

# Permutation Feature Importance
importance_result = permutation_importance(
    model, X_test_corrected, y_test, n_repeats=10, random_state=42, scoring='r2'
)
```

## 6.3 :- Tkinter Desktop GUI Prediction Function

```
def predict_marks():
    try:
        values = {}
        for feature in FEATURES:
            raw_value = entries[feature].get().strip()
            if not raw_value: raise ValueError(f'Please enter {feature}.')
            val = float(raw_value)
            min_val, max_val = INPUT_LIMITS[feature]
```

```
            if not min_val <= val <= max_val:
                raise ValueError(f'{feature} must be between {min_val} and {max_val}.')
            values[feature] = val

        new_df = pd.DataFrame([values], columns=FEATURES)
        new_df = new_df.clip(lower=lower_bounds, upper=upper_bounds, axis='columns')
        pred = float(np.clip(model.predict(new_df)[0], 0, 100))
        result_label.config(text=f'Predicted Final Marks: {pred:.2f} / 100')
    except ValueError as err:
        messagebox.showerror('Invalid Input', str(err))
```

## 7. TESTING AND DEBUGGING

Testing and debugging are indispensable stages in software development and applied machine learning. These processes ensure that mathematical transformations function reliably, that models make accurate predictions, and that the desktop GUI validates inputs without runtime exceptions.

### Types of Testing

**1. Unit Testing:** Testing individual functions such as IQR calculation, median imputation, and bounds clipping to verify accurate isolated outputs.

**2. Integration Testing:** Validating data transfer between the preprocessed Pandas DataFrames, Scikit-learn regressors, and the Tkinter GUI form widgets.

**3. System Testing:** Executing the end-to-end Python pipeline from dataset ingestion and figure generation to GUI invocation.

**4. User Acceptance Testing:** Inviting sample users to enter diverse student scores, validating ease of use, responsiveness, and clear error dialogs.

### TEST CASES:--

| Test Id | Description | Input | Expected Result | Actual Output | Status |
|---|---|---|---|---|---|
| TC-1 | Dataset Ingestion & Schema Check | Final_Marks_Data (1).csv | 2000 records loaded, columns verified | As expected | **Pass** |
| TC-2 | Missing Value Median Imputation | Columns with NaN values | Impute training medians accurately | As expected | **Pass** |
| TC-3 | IQR Outlier Capping (Winsorization) | Feature values > upper bound | Values clipped to learned bounds | As expected | **Pass** |
| TC-4 | Prediction with Valid Inputs | Att:85, T1:32, T2:34, Assg:8, Hrs:3 | Predicted marks & category shown | As expected | **Pass** |
| TC-5 | Empty Input Field Validation | Leave Internal Test 1 empty | Error modal: 'Please enter feature' | As expected | **Pass** |
| TC-6 | Out of Bounds Input Validation | Enter 55 in Test 1 (max 40) | Error modal: 'Must be between 0-40' | As expected | **Pass** |
| TC-7 | Non-numeric Input Validation | Enter 'ninety' in Attendance | Error modal: 'Must be a finite number' | As expected | **Pass** |

| | | | | | |
|---|---|---|---|---|---|
| TC-8 | GUI Clear / Reset Functionality | Click 'Clear' button | Entries emptied, score reset to '--' | As expected | **Pass** |

## 8. CONCLUSION AND FUTURE ENHANCEMENT

The Student Performance Prediction System is designed to simplify and improve the process of academic monitoring and intervention by providing teachers and students with an accurate, explainable desktop application to forecast semester marks. Traditional evaluation systems calculate outcomes only retrospectively after final exams, offering no opportunity for early assistance. This system provides an effective machine learning solution built on Multiple Linear Regression that estimates final scores in real time based on continuous formative evaluation metrics.

The system enables educators to understand which pedagogical factors exert the greatest mathematical impact on final examination scores. By using Scikit-Learn, Pandas, NumPy, Matplotlib, and Tkinter, the system reinforces core concepts of data engineering, leakage-free outlier Winsorization, model diagnostics, permutation feature importance, and interactive GUI development.

Overall, the Student Performance Prediction System improves academic intervention efficiency by providing transparent forecasts (82.92% $R^2$), rigorous bounds validation, and responsive qualitative risk classification for institutional stakeholders.

### Future Enhancements

**1. Non-Linear Ensemble Models:** Incorporating Random Forest Regressor and Gradient Boosting (XGBoost) to capture complex non-linear factor interactions.

**2. Web & Mobile Deployment:** Migrating the Tkinter desktop GUI to a cloud-hosted full-stack web application using FastAPI and React JS for cross-device accessibility.

**3. Relational Database Integration:** Connecting the application to PostgreSQL or MongoDB with role-based access for students, teachers, and administrators.

**4. Automated Early Warning Notifications:** Configuring automated email and SMS alerts to notify faculty advisors whenever a student is classified as 'At Risk'.

**5. Multi-Class Letter Grade Classification:** Extending the regression model to output official institutional letter grades (O, E, A, B, C, D, F) alongside SGPA estimates.

**6. Multi-Factor Feature Expansion:** Integrating Learning Management System (LMS) engagement logs, library visits, and laboratory continuous assessment scores.

## 9. REFERENCES :--

**Scikit-learn – Machine Learning in Python**
https://scikit-learn.org
(For Multiple Linear Regression, train_test_split, evaluation metrics, and permutation importance)

**Pandas – Python Data Analysis Library**
https://pandas.pydata.org
(For DataFrame structures, CSV ingestion, and median imputation)

**NumPy – Scientific Computing in Python**
https://numpy.org
(For array operations, numeric clipping, and mathematical functions)

**Matplotlib – Visualization with Python**
https://matplotlib.org
(For generating distribution histograms, scatter diagrams, and residual plots)

**Seaborn – Statistical Data Visualization**
https://seaborn.pydata.org
(For correlation matrix heatmaps and styled distribution plots)

**Python Tkinter – Graphical User Interface Library**
https://docs.python.org/3/library/tkinter.html
(For building desktop windows, TTK form controls, and validation dialogs)

**Educational Data Mining Literature**
Romero & Ventura (2010), IEEE Transactions on SMC
(For background literature on student performance predictive indicators)

**Class notes and study materials provided by Brainware University**
Department of Computational Science, Brainware University, Barasat, Kolkata